

Medical Data History

SQL Data Analysis Project

Ajith Kumar
Project ID: PRSQL-02

Project Overview

This project uses SQL to analyze a healthcare dataset covering patients, doctors, hospital admissions, and provinces. The dataset was accessed through a remote MySQL server using DBeaver, and the goal was to answer 34 real-world business questions using SQL alone, filtering, sorting, joins across tables, string and date handling, and grouping with aggregate functions.

The database has four connected tables: patients, doctors, admissions, and province_names. Patients link to provinces through a province code, and admissions link patients to the doctors who treated them.

Each question below is shown with its query, a plain-English explanation of what the query does and why it is written that way, and a snapshot of the real result returned when it was run.

Key Skills Demonstrated

- Filtering with WHERE, including text patterns, ranges, and value lists
- Handling missing data correctly with IS NULL and COALESCE
- Aggregate functions: COUNT, SUM, MAX, MIN, with and without GROUP BY
- GROUP BY across single and multiple columns, including calculated values
- HAVING versus WHERE, filtering groups after aggregation versus rows before it
- Multi-key sorting with ORDER BY
- String functions: CONCAT, UPPER, LOWER, LENGTH
- Date functions: YEAR, DAY, and date range filtering
- CASE WHEN conditional logic for flags and pivoted counts
- INNER JOIN across two and three related tables
- UNION ALL to stack results from different tables
- Reading a schema, understanding keys, and adjusting a column with ALTER TABLE
- Working around real-world limits such as read-only database access

1

Question 1

Show first name, last name, and gender of patients whose gender is 'M'.

```
SELECT first_name, last_name, gender FROM patients WHERE gender = 'M';
```

This pulls three columns and keeps only the rows where gender equals M. A WHERE clause is just a filter, it decides which rows get through before anything else happens.

#	first_name	last_name	gender
1	Donald	Waterfield	M
2	Mickey	Baasha	M
3	Jiji	Sharma	M
4	Blair	Diaz	M
5	Charles	Wolfe	M
6	Thomas	ONeill	M
7	Sonny	Beckett	M
8	Cedric	Coltrane	M
9	Hank	Spencer	M
10	Rick	Bennett	M

2

Question 2

Show first name and last name of patients who do NOT have allergies.

```
SELECT first_name, last_name FROM patients WHERE allergies IS NULL;
```

Missing data in SQL is stored as NULL, not as an empty box or a zero. You cannot check for it with a normal equals sign, you have to use IS NULL. This query finds every patient whose allergy field was left blank.

#	first_name	last_name
1	Donald	Waterfield
2	Blair	Diaz
3	Thomas	ONeill
4	Sonny	Beckett
5	Cedric	Coltrane
6	Hank	Spencer
7	Sara	di Marco
8	Amy	Leela
9	Rachel	Winterbourne
10	John	West

3

Question 3

Show first name of patients that start with the letter 'C'.

```
SELECT first_name FROM patients WHERE first_name LIKE 'C%';
```

LIKE is used for pattern matching instead of exact matching. The percent sign is a wildcard meaning any characters can follow, so 'C%' means the name has to start with C and can be any length after that.

#	first_name
1	Charles
2	Cedric
3	Charles
4	Cross
5	Calleigh
6	Catherine
7	Caroline
8	Casanova
9	Chen
10	Charmian

4

Question 4

Show first name and last name of patients that weigh within the range of 100 to 120 (inclusive).

```
SELECT first_name, last_name FROM patients WHERE weight BETWEEN 100 AND 120;
```

BETWEEN checks if a number falls inside a range, and both the low and high ends are included. It is really just a shorter way of writing `weight >= 100 AND weight <= 120`.

#	first_name	last_name
1	Jiji	Sharma
2	Blair	Diaz
3	Thomas	ONeill
4	Sonny	Beckett
5	Tom	Halliwell
6	Jon	Doggett
7	Angel	Edwards
8	John	Farley
9	Temple	Russert
10	Don	Edwards

5

Question 5

Update the patients table so that if allergies is NULL, it gets replaced with 'NKA'.

```
UPDATE patients SET allergies = 'NKA' WHERE allergies IS NULL;
```

UPDATE changes data that already exists, rather than just reading it, which makes the WHERE clause critical. Leave it off and every single row gets overwritten. In this case the database account only had read access to this table, so the UPDATE could not actually run. A SELECT using COALESCE(allergies, 'NKA') was used instead to check what the result would have looked like, without changing anything.

#	first_name	last_name	allergies
1	Donald	Waterfield	NKA
2	Mickey	Baasha	Sulfa
3	Jiji	Sharma	Penicillin
4	Blair	Diaz	NKA
5	Charles	Wolfe	Penicillin
6	Sue	Falcon	Penicillin
7	Thomas	ONeill	NKA
8	Sonny	Beckett	NKA
9	Sister	Spitzer	Penicillin
10	Cedric	Coltrane	NKA

6

Question 6

Show first name and last name concatenated into one column as full name.

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM patients;
```

CONCAT glues text values together into one string, and the AS keyword just renames the resulting column so it reads cleanly.

#	full_name
1	Donald Waterfield
2	Mickey Baasha
3	Jiji Sharma
4	Blair Diaz
5	Charles Wolfe
6	Sue Falcon
7	Thomas ONeill
8	Sonny Beckett
9	Sister Spitzer
10	Cedric Coltrane

7

Question 7

Show first name, last name, and the full province name of each patient.

```
SELECT p.first_name, p.last_name, pn.province_name
FROM patients p
JOIN province_names pn ON p.province_id = pn.province_id;
```

The patients table only stores a short province code, not the full name, so this pulls it from a second table using a JOIN matched on that shared code. Giving each table a short alias (p, pn) just saves typing the full table name over and over.

#	first_name	last_name	province_name
1	Donald	Waterfield	Ontario
2	Mickey	Baasha	Ontario
3	Jiji	Sharma	Ontario
4	Blair	Diaz	Ontario
5	Charles	Wolfe	Ontario
6	Sue	Falcon	Ontario
7	Thomas	ONeill	Nova Scotia
8	Sonny	Beckett	Ontario
9	Sister	Spitzer	Ontario
10	Cedric	Coltrane	Ontario

8

Question 8

Show how many patients have a birth_date with 2010 as the birth year.

```
SELECT COUNT(*) AS total_patients
FROM patients
WHERE YEAR(birth_date) = 2010;
```

YEAR() pulls just the year out of a full date, and COUNT(*) tallies how many rows match after that filter is applied.

#	total_patients
1	55

9

Question 9

Show the first_name, last_name, and height of the patient with the greatest height.

```
SELECT first_name, last_name, height
FROM patients
ORDER BY height DESC
LIMIT 1;
```

Sorting tallest to shortest and keeping only the first row is the standard way to find the record that holds an extreme value. MAX(height) alone would only give the number, not who it belongs to.

#	first_name	last_name	height
1	Sam	Haruko	226

10

Question 10

Show all columns for patients who have one of these patient_ids: 1, 45, 534, 879, 1000.

```
SELECT *  
FROM patients  
WHERE patient_id IN (1, 45, 534, 879, 1000);
```

IN() checks a value against a list, which is a shorter way of writing several OR conditions strung together.

#	patient_id	first_name	last_name	city
1	1	Donald	Waterfield	Barrie
2	45	Cross	Gordon	Ancaster
3	534	Don	Zatara	Timmins
4	879	Orla	Shawn	Sarnia
5	1000	Rick	Williams	Hamilton

11

Question 11

Show the total number of admissions.

```
SELECT COUNT(*) AS total_admissions FROM admissions;
```

A plain row count across the whole table, with no filter attached.

#	total_admissions
1	5067

12

Question 12

Show all columns from admissions where the patient was admitted and discharged on the same day.

```
SELECT *  
FROM admissions  
WHERE admission_date = discharge_date;
```

This compares two columns from the same row against each other, instead of comparing a column to a fixed value.

#	patient_id	admission_date	discharge_date	diagnosis
1	1	2018-09-20	2018-09-20	Ineffective Breathing Pattern
2	9	2018-12-31	2018-12-31	Ruptured Appendicitis
3	10	2019-02-27	2019-02-27	Lower Quadrant Pain
4	17	2019-03-04	2019-03-04	Diabetes Mellitus
5	28	2019-03-30	2019-03-30	Cancer Of The Stomach

13

Question 13

Show the total number of admissions for patient_id 579.

```
SELECT COUNT(*) AS total_admissions  
FROM admissions  
WHERE patient_id = 579;
```

Same COUNT idea as before, just scoped down to one specific patient with a WHERE filter.

#	total_admissions
1	2

14

Question 14

Based on the cities patients live in, show unique cities that are in province_id 'NS'.

```
SELECT DISTINCT city
FROM patients
WHERE province_id = 'NS';
```

DISTINCT strips out duplicates from the result, so each city shows up once no matter how many patients live there.

#	city
1	Port Hawkesbury
2	Halifax
3	Inverness

15

Question 15

Find the first_name, last_name, and birth_date of patients who have height more than 160 and weight more than 70.

```
SELECT first_name, last_name, birth_date
FROM patients
WHERE height > 160 AND weight > 70;
```

AND means both conditions have to be true at once for a row to make it through. If either one fails, the row is dropped.

#	first_name	last_name	birth_date
1	Alice	Donovan	1949-06-27
2	Phoebe	Bigelow	2002-03-17
3	Rose	Spellman	1957-05-31
4	Calleigh	Dean	2003-07-06
5	Vicki	Babbling	2003-08-10
6	Ernest	Clarkson	1962-11-22
7	Frank	Capone	2004-02-15
8	Beanie	Nelson	1993-06-11
9	Henry	Hernandez	1954-03-30
10	Lyan	Russo	2005-05-15

16

Question 16

Show unique birth years from patients and order them ascending.

```
SELECT DISTINCT YEAR(birth_date) AS birth_year
FROM patients
ORDER BY birth_year ASC;
```

This combines DISTINCT with a date function and an explicit sort direction, so it lists every birth year that appears at least once, earliest first.

#	birth_year
1	1918
2	1923
3	1925
4	1926
5	1927
6	1928
7	1929
8	1931
9	1933
10	1934

17

Question 17

Show unique first names from the patients table which only occur once in the list.

```
SELECT first_name
FROM patients
GROUP BY first_name
HAVING COUNT(*) = 1;
```

GROUP BY clusters every row that shares the same first name, COUNT(*) counts how many rows landed in each cluster, and HAVING keeps only the clusters where that count is exactly one. HAVING filters groups after they are formed, which is different from WHERE, which filters individual rows before grouping happens.

#	first_name
1	Abby
2	Adelaide
3	Adelia
4	Akira
5	Albert
6	Aldo
7	Alec
8	Alicia
9	Allan
10	Alpa

18

Question 18

Show patient_id and first_name from patients where their first_name starts and ends with 's' and is at least 6 characters long.

```
SELECT patient_id, first_name
FROM patients
WHERE first_name LIKE 's%s' AND LENGTH(first_name) >= 6;
```

This stacks two conditions together, a LIKE pattern that checks the first and last letter, plus a LENGTH() check to make sure the name is not too short.

#	patient_id	first_name
1	496	Spiros
2	629	Spiros
3	648	Stanislaus
4	1273	Stanislaus
5	1789	Seamus
6	1926	Stanislaus
7	1996	Stanislaus
8	2258	Spiros
9	2378	Stanislaus
10	2771	Stanislaus

19

Question 19

Show patient_id, first_name, last_name from patients whose diagnosis is 'Dementia'. Primary diagnosis is stored in the admissions table.

```
SELECT p.patient_id, p.first_name, p.last_name
FROM patients p
JOIN admissions a ON p.patient_id = a.patient_id
WHERE a.diagnosis = 'Dementia';
```

A JOIN is needed here because the column being filtered on, diagnosis, lives in a different table than the columns being displayed.

#	patient_id	first_name	last_name
1	160	Miranda	Delacour
2	178	David	Bustamante
3	207	Matt	Celine
4	613	Jaki	Granger
5	836	Montana	Vimes
6	924	Simon	Spellman
7	1201	Irene	Murphy
8	1264	Jillian	Valentine
9	1402	Kathryn	Hallow
10	1491	Doris	McGrew

Question 20

Display every patient's first_name, ordered by the length of the name and then alphabetically.

```
SELECT first_name
FROM patients
ORDER BY LENGTH(first_name), first_name;
```

You can sort by more than one thing at once. The second key, first_name, only kicks in to break ties when two names are exactly the same length.

#	first_name
1	Al
2	Al
3	Al
4	Al
5	Al
6	Al
7	Bo
8	Bo
9	Bo
10	Bo

Question 21

Show the total amount of male patients and total amount of female patients in the patients table, displayed in the same row.

```
SELECT
    SUM(CASE WHEN gender = 'M' THEN 1 ELSE 0 END) AS male_count,
    SUM(CASE WHEN gender = 'F' THEN 1 ELSE 0 END) AS female_count
FROM patients;
```

CASE WHEN works like an if or else check inside a query. It turns each row into a 1 or a 0 depending on gender, and SUM adds those up, which gives both totals side by side in a single row instead of two separate rows the way a plain GROUP BY gender would.

(No clean result screenshot available for this query in the source material.)

23

Question 23

Show patient_id and diagnosis from admissions, finding patients admitted multiple times for the same diagnosis.

```
SELECT patient_id, diagnosis
FROM admissions
GROUP BY patient_id, diagnosis
HAVING COUNT(*) > 1;
```

Grouping by two columns at once means a group only forms when both values match on the same rows. That captures same patient, same diagnosis, rather than just any repeat admission.

#	patient_id	diagnosis
1	137	Pregnancy
2	320	Pneumonia
3	1577	Congestive Heart Failure
4	2004	Left Shoulder Rotator Cuff Repair
5	2859	Severed Spine At C3
6	3012	Appendicitis
7	3367	Pyelonephritis
8	3468	Congestive Heart Failure
9	4083	Congestive Heart Failure
10	4121	Congestive Heart Failure

Question 24

Show the city and the total number of patients in it, ordered most to least, then by city name.

```
SELECT city, COUNT(*) AS total_patients
FROM patients
GROUP BY city
ORDER BY total_patients DESC, city ASC;
```

A two level sort, the main order is by patient count from high to low, and city name only steps in as a tiebreaker when two cities have the exact same count.

#	city	total_patients
1	Linton	1
2	Rockford	1
3	Uretersburg	1
4	Brandon	2
5	Georgetown	2
6	Red Deer	2
7	Scarborough	2
8	Stainer	2
9	Tuscorora	2
10	Unionville	2

Question 25

Show first name, last name, and role of every person who is either a patient or a doctor.

```
SELECT first_name, last_name, 'Patient' AS role FROM patients
UNION ALL
SELECT first_name, last_name, 'Doctor' AS role FROM doctors;
```

UNION ALL stacks the results of two separate SELECT statements on top of each other into one list, which is different from a JOIN that lines tables up side by side. The literal text 'Patient' and 'Doctor' just label which table each row came from.

#	first_name	last_name	role
1	Donald	Waterfield	Patients
2	Mickey	Baasha	Patients
3	Jiji	Sharma	Patients
4	Blair	Diaz	Patients
5	Charles	Wolfe	Patients
6	Sue	Falcon	Patients
7	Thomas	ONeill	Patients
8	Sonny	Beckett	Patients
9	Sister	Spitzer	Patients
10	Cedric	Coltrane	Patients

Question 26

Show all allergies ordered by how common they are, with NULL values removed.

```
SELECT allergies, COUNT(*) AS total
FROM patients
WHERE allergies IS NOT NULL
GROUP BY allergies
ORDER BY total DESC;
```

This filters out the blank allergy entries first, then groups what is left and ranks each allergy by how often it shows up.

#	allergies	total
1	Penicillin	1082
2	Codeine	305
3	Sulfa	157
4	ASA	99
5	Sulfa Drugs	71
6	Peanuts	52
7	Iodine	48
8	Tylenol	42
9	Bee Stings	40
10	Valporic Acid	38

Question 27

Show all patients born in the 1970s, sorted from the earliest birth date.

```
SELECT first_name, last_name, birth_date
FROM patients
WHERE birth_date BETWEEN '1970-01-01' AND '1979-12-31'
ORDER BY birth_date ASC;
```

As long as dates are written in YYYY-MM-DD format, SQL can compare and sort them the same way it compares plain numbers.

#	first_name	last_name	birth_date
1	Frances	Kobayakawa	1970-01-02
2	Sunny	Burrell	1970-01-07
3	Penelope	Beckett	1970-01-14
4	Deborah	Stewart	1970-01-14
5	Augusta	Decker	1970-01-22
6	Sookie	Brearly	1970-02-01
7	Temple	Wylie	1970-02-10
8	Deanna	Spano	1970-03-23
9	Jadu	Principal	1970-03-28
10	Betty	Stephens	1970-03-28

Question 28

Display each patient's full name as LAST,first (uppercase last name, lowercase first name), ordered by first name descending.

```
SELECT CONCAT(UPPER(last_name), ', ', LOWER(first_name)) AS full_name
FROM patients
ORDER BY first_name DESC;
```

UPPER, LOWER and CONCAT are combined to build one exact text format. Note that the sort still refers to the original first_name column, not the reformatted output.

#	full_name
1	MILLER,zoe
2	CORBIE,ziva
3	KOBAYAKAWA,zenigata
4	OVERSTREET,zenigata
5	BENNETT,zen
6	MEPHESTO,zelda
7	MORRIS,zelda
8	THOMAS,zefram
9	FLUTE,zefram
10	TYRELL,zatanna

Question 29

Show the province_id(s) and sum of height, where the total height of a province's patients is 7,000 or more.

```
SELECT province_id, SUM(height) AS total_height
FROM patients
GROUP BY province_id
HAVING SUM(height) >= 7000;
```

HAVING is not limited to COUNT, it works with any aggregate function. Here it filters groups based on a SUM total instead.

#	province_id	total_height
1	BC	7720
2	NS	9765
3	ON	678037

Question 30

Show the difference between the largest and smallest weight for patients with the last name 'Maroni'.

```
SELECT MAX(weight) - MIN(weight) AS weight_difference  
FROM patients  
WHERE last_name = 'Maroni';
```

MAX and MIN are aggregate functions. With no GROUP BY at all, the entire filtered result is treated as one single group, so you just get one number back.

#	weight_difference
1	71

Question 31

Show all days of the month (1-31) and how many admissions happened on that day, sorted most to least.

```
SELECT DAY(admission_date) AS day_of_month, COUNT(*) AS total_admissions
FROM admissions
GROUP BY DAY(admission_date)
ORDER BY total_admissions DESC;
```

DAY() pulls out just the day of month number from a date. Worth noting, this only lists days that actually had at least one admission, a day with zero admissions would need a separate reference list joined in to show up as zero.

#	day_of_month	total_admissions
1	11	184
2	4	184
3	9	183
4	2	180
5	12	179
6	6	179
7	16	177
8	21	174
9	28	173
10	13	173

Question 32

Show all patients grouped into weight groups of 10 (e.g. 100 to 109 becomes 100), with total patients per group, ordered highest group first.

```
SELECT FLOOR(weight / 10) * 10 AS weight_group, COUNT(*) AS total_patients
FROM patients
GROUP BY weight_group
ORDER BY weight_group DESC;
```

FLOOR always rounds down, which is what correctly buckets a weight like 119 into the 110 group instead of accidentally rounding it up into 120. A calculated value like this can be grouped on just like a real column.

#	weight_group	total_patients
1	140	6
2	130	59
3	120	191
4	110	426
5	100	507
6	90	403
7	80	478
8	70	633
9	60	685
10	50	443

33

Question 33

Show patient_id, weight, height, and isObese (as 0 or 1) from patients. Obese is defined as weight in kg divided by height in meters.

```
SELECT patient_id, weight, height,  
       CASE WHEN weight / (height / 100.0) > 25 THEN 1 ELSE 0 END AS isObese  
FROM patients;
```

Height gets converted from centimeters to meters before applying the formula. CASE WHEN then turns that number into a simple 1 or 0 flag based on whether it crosses the threshold.

#	patient_id	weight	height	isObese
1	1	65	156	1
2	2	76	185	1
3	3	106	194	1
4	4	104	191	1
5	5	10	47	0
6	6	5	43	0
7	7	117	180	1
8	8	105	174	1
9	9	95	173	1
10	10	61	157	1

34

Question 34

Show patient_id, first_name, last_name, and attending doctor's specialty, for patients diagnosed with 'Epilepsy' where the doctor's first name is 'Lisa'.

```
SELECT p.patient_id, p.first_name, p.last_name, d.speciality
FROM patients p
JOIN admissions a ON p.patient_id = a.patient_id
JOIN doctors d ON a.doctor_id = d.doctor_id
WHERE a.diagnosis = 'Epilepsy' AND d.first_name = 'Lisa';
```

A three table join, chaining patients to admissions to doctors through their shared keys. This also needed a small schema fix beforehand, the admissions table originally called its doctor column attending_doctor_id, which had to be renamed to doctor_id so it would line up with the doctors table.

#	patient_id	first_name	last_name	speciality
1	468	Frank	Anderson	Obstetrician/Gynecologist
2	701	Precious	Ashton	Obstetrician/Gynecologist